Пояснения к программе search.cpp

Практическая работа «Алгоритмы поиска», вариант 8

1 Общая идея программы

Задача: найти заданный элемент в неотсортированном массиве символьных строк двумя способами — последовательным поиском и интерполирующим поиском, затем сравнить их по времени и памяти.

Последовательный поиск работает с массивом как есть. **Интерполирующий поиск** требует отсортированного массива, поэтому программа сначала делает отсортированную копию. Чтобы применить формулу интерполяции к строкам, каждая строка переводится в число функцией key.

Программа состоит из четырёх функций: sequentialSearch, key, interpolationSearch, readFile — и функции main, которая работает в двух режимах: поиск одной строки или прогон тестов из архива с замером времени.

Сборка: clang++ -std=c++17 -O2 -o search search.cpp (или g++ с теми же ключами).

2 Построчный разбор

В таблицах указан номер строки в файле search.cpp, сама строка и её пояснение. Пустые строки пропущены.

2.1 Подключение библиотек

Подключаются стандартные библиотеки C++, нужные программе.

№	Код	Пояснение
1	`#include <iostream>`	Подключает потоки ввода-вывода: cout для вывода на экран.
2	`#include <fstream>`	Подключает работу с файлами: ifstream для чтения тестовых файлов.
3	`#include <vector>`	Подключает vector — динамический массив, в котором хранятся строки.
4	`#include <string>`	Подключает string — тип символьной строки.

№	Код	Пояснение
5	`#include <algorithm>`	Подключает алгоритмы стандартной библиотеки, нам нужна сортировка sort.
6	`#include <chrono>`	Подключает chrono — библиотеку для точного измерения времени.
7	`#include <cstdio>`	Подключает printf для форматированного вывода строки таблицы.
8	`using namespace std;`	Позволяет писать vector, string, cout без префикса std::.
9	`using namespace std::chrono;`	Позволяет писать steady_clock, duration без префикса std::chrono::.

2.2 Функция sequentialSearch — последовательный поиск

Просматривает массив с начала до конца и сравнивает каждый элемент с искомой строкой. Работает с неотсортированным массивом. Время O(N), дополнительная память O(1).

№	Код	Пояснение
11	`int sequentialSearch(const vector<string>& a, const string& x) {`	Функция последовательного поиска. Принимает массив a и искомую строку x по константной ссылке (без копирования). Возвращает индекс или −1.
12	`    for (int i = 0; i < (int)a.size(); i++)`	Цикл по всем индексам массива от 0 до N−1. (int) приводит размер к типу int, чтобы сравнивать с i без предупреждений.
13	`        if (a[i] == x) return i;`	Сравниваем текущий элемент с искомой строкой. Если совпали — сразу возвращаем индекс i (первое вхождение).
14	`    return -1;`	Цикл закончился, совпадений нет — возвращаем −1 («не найдено»).
15	`}`	Конец функции sequentialSearch.

2.3 Функция key — числовой ключ строки

Формула интерполяции работает с числами, а у нас строки. Функция key превращает строку в число так, чтобы строка, стоящая дальше по алфавиту, получала ключ не меньше. Первые 6 символов рассматриваются как цифры числа в системе счисления по основанию 256 (код символа — от 0 до 255). Например, key("ab") = $97 \cdot 256^5 + 98 \cdot 256^4$ (97 и 98 — коды букв a и b, недостающие символы — нули).

№	Код	Пояснение
17	`unsigned long long key(const string& s) {`	Функция перевода строки в число. unsigned long long — беззнаковое 64-битное целое, вмещает 6 символов по 8 бит (48 бит).
18	`    unsigned long long k = 0;`	Начальное значение ключа — 0.
19	`    for (int i = 0; i < 6; i++)`	Обрабатываем ровно 6 первых позиций строки.
20	`        k = k * 256 + (i < (int)s.size() ? (unsigned char)s[i] : 0);`	Сдвигаем накопленное число на один «разряд» (умножаем на 256) и добавляем код символа. Если строка короче 6 символов, добавляем 0. (unsigned char) — чтобы код символа был от 0 до 255 и не стал отрицательным.
21	`    return k;`	Возвращаем полученный ключ.
22	`}`	Конец функции key.

2.4 Функция interpolationSearch — интерполирующий поиск

Работает с отсортированным массивом. В отличие от двоичного поиска, который всегда делит область пополам, интерполирующий поиск «прикидывает», где должен находиться элемент, исходя из его значения — как человек, открывающий словарь на букву «т» ближе к концу. Позиция вычисляется по формуле pos = low + (key(x) − key(a[low])) / (key(a[high]) − key(a[low])) · (high − low). В среднем O(log log N) шагов при равномерных данных, в худшем случае O(N).

№	Код	Пояснение
24	`int interpolationSearch(const vector<string>& a, const string& x) {`	Функция интерполирующего поиска. Массив a должен быть отсортирован. Возвращает индекс найденного элемента в a или −1.
25	`    int low = 0, high = (int)a.size() - 1;`	Границы области поиска: low — левая, high — правая. В начале это весь массив.
26	`    unsigned long long kx = key(x);`	Один раз вычисляем ключ искомой строки kx.
27	`    while (low <= high) {`	Повторяем, пока область поиска не пуста.
28	`        unsigned long long kl = key(a[low]), kh = key(a[high]);`	Ключи элементов на левой (kl) и правой (kh) границах области.
29	`        if (kx < kl || kx > kh) break;`	Если ключ искомой строки меньше левого или больше правого, строки x в области быть не может — выходим из цикла.
30	`        int pos = low;`	Позиция по умолчанию — левая граница (на случай, если kh = kl и делить нельзя).

№	Код	Пояснение
31	`if (kh != kl) pos = low + (int)((long double)(kx - kl) / (kh - kl) * (high - low));`	Формула интерполяции: (kx − kl) / (kh − kl) — какую долю диапазона ключей занимает kx; умножаем её на длину области (high − low) и прибавляем к low. long double — деление в дробных числах. Проверка kh != kl исключает деление на ноль.
32	`if (a[pos] == x) return pos;`	Если элемент в позиции pos равен искомому — нашли, возвращаем pos.
33	`if (a[pos] < x) low = pos + 1;`	Если элемент меньше искомого (по алфавиту), искомый правее — сдвигаем левую границу за pos.
34	`else high = pos - 1;`	Иначе элемент больше искомого — искомый левее, сдвигаем правую границу перед pos.
35	`}`	Конец цикла while.
36	`return -1;`	Область поиска опустела или x вне диапазона — элемента нет, возвращаем −1.
37	`}`	Конец функции interpolationSearch.

2.5 Функция readFile — чтение тестового файла

Читает файл в формате задания: в первой строке количество элементов N, в следующих N строках — элементы.

№	Код	Пояснение
39	`vector<string> readFile(const string& name) {`	Функция чтения массива из файла с именем name. Возвращает вектор строк.
40	`ifstream in(name);`	Открываем файл для чтения.
41	`int n = 0;`	Переменная для количества элементов; 0 — на случай, если файл не открылся.
42	`in >> n;`	Читаем первое число файла — N.
43	`vector<string> a(n);`	Создаём массив из n пустых строк.
44	`for (auto& s : a) in >> s;`	Читаем в каждую строку массива очередное слово файла. Оператор >> пропускает пробелы и переводы строк (включая \r из Windows-файлов).
45	`return a;`	Возвращаем заполненный массив.
46	`}`	Конец функции readFile.

2.6 Функция main — режим поиска одной строки

Запуск: ./search tests/ucba1.txt svrf. Ищет заданную строку обоими способами и выводит индексы.

№	Код	Пояснение
48	`int main(int argc, char* argv[]) {`	Главная функция. argc — количество аргументов командной строки, argv — сами аргументы (argv[0] — имя программы).
49	`    if (argc == 3) {`	Если передано два аргумента (файл и строка), работаем в режиме поиска одной строки.
50	`        vector<string> a = readFile(argv[1]);`	Читаем массив из файла argv[1].
51	`        vector<string> s = a;`	Делаем копию массива для интерполирующего поиска (исходный массив не трогаем).
52	`        sort(s.begin(), s.end());`	Сортируем копию по алфавиту — без этого интерполирующий поиск не работает.
53	`        cout << "Последовательный: " << sequentialSearch(a, argv[2]) << "\n";`	Последовательный поиск строки argv[2] в исходном массиве, выводим индекс (−1 — не найдено).
54	`        cout << "Интерполирующий (индекс в отсортированном): " << interpolationSearch(s, argv[2]) << "\n";`	Интерполирующий поиск в отсортированной копии. Индекс относится к отсортированному массиву, поэтому может отличаться от индекса последовательного поиска.
55	`        return 0;`	Завершаем программу.
56	`    }`	Конец блока if.

2.7 Функция main — прогон тестов и измерения

Запуск: ./search (папка tests в текущей директории) или ./search путь/к/tests. Для каждого тестового файла варианта выполняет 1000 поисков каждым способом, проверяет правильность результатов и печатает строку таблицы: время одного поиска, время сортировки, дополнительную память и результат проверки.

№	Код	Пояснение
58	`    string dir = argc == 2 ? argv[1] : "tests";`	Папка с тестами: из аргумента, если он передан, иначе "tests".
59	`    const int Q = 1000;`	Q = 1000 — сколько поисков выполняется на каждом файле, по ним усредняется время.
60	`    cout << "файл            N  посл.(мкс)  сорт.(мс)  интерп.(мкс)  доп.память(байт)  проверка\n";`	Печатаем заголовок таблицы результатов.
61	`    for (string group : {"ucba", "ucbb", "ucbc"})`	Перебираем три группы тестовых файлов: ucba, ucbb, ucbc.
62	`        for (int t = 1; t <= 20; t++) {`	В каждой группе — файлы с номерами от 1 до 20.

№	Код	Пояснение
63	`string name = group + to_string(t) + ".txt";`	Составляем имя файла, например "ucba1.txt".
64	`vector<string> a = readFile(dir + "/" + name);`	Читаем массив из файла папка/имя.
65	`if (a.empty()) continue;`	Если файла нет (массив пустой) — пропускаем его.
66	`int n = a.size();`	Запоминаем размер массива n.
67	`bool ok = true;`	Флаг правильности: станет false, если хотя бы один поиск ошибётся.
69	`auto t0 = steady_clock::now();`	Засекаем время начала последовательных поисков.
70	`for (int i = 0; i < Q; i++) {`	Выполняем Q последовательных поисков.
71	`string x = a[(long long)i * n / Q];`	Искомый элемент — элемент массива с индексом i·n/Q: так запросы равномерно распределены по всему массиву. long long — чтобы произведение не переполнилось.
72	`int r = sequentialSearch(a, x);`	Ищем x последовательным поиском.
73	`ok = ok && r >= 0 && a[r] == x;`	Проверка: элемент должен найтись (r ≥ 0), и по найденному индексу должна стоять именно строка x.
74	`}`	Конец цикла последовательных поисков.
75	`auto t1 = steady_clock::now();`	Засекаем время окончания: t1 − t0 — время Q последовательных поисков.
76	`vector<string> s = a;`	Копия массива для интерполирующего поиска — это и есть дополнительная память O(N).
77	`sort(s.begin(), s.end());`	Сортируем копию: время t2 − t1 — время подготовки O(N·log N).
78	`auto t2 = steady_clock::now();`	Засекаем время после сортировки.
79	`for (int i = 0; i < Q; i++) {`	Выполняем Q интерполирующих поисков.
80	`string x = a[(long long)i * n / Q];`	Те же искомые элементы, что и для последовательного поиска.
81	`int r = interpolationSearch(s, x);`	Ищем x интерполирующим поиском в отсортированной копии s.
82	`ok = ok && r >= 0 && s[r] == x;`	Проверка: элемент найден, и в s по найденному индексу стоит строка x.
83	`}`	Конец цикла интерполирующих поисков.
84	`auto t3 = steady_clock::now();`	Засекаем время: t3 − t2 — время Q интерполирующих поисков.
86	`string absent = a[0] + "#";`	Строка, которой точно нет в массиве: первый элемент плюс символ "#" (в тестах только латинские буквы).
87	`ok = ok && sequentialSearch(a, absent) == -1 && interpolationSearch(s, absent) == -1;`	Проверка: оба поиска должны вернуть −1 для отсутствующей строки.
89	`printf("%-11s %6d %11.3f %10.3f %13.3f %17zu  %s\n", name.c_str(), n,`	Печатаем строку таблицы: имя файла и n.
90	`duration<double, micro>(t1 - t0).count() / Q,`	Среднее время одного последовательного поиска в микросекундах (общее время / Q).

№	Код	Пояснение
91	`duration<double, milli>(t2 - t1).count(),`	Время сортировки в миллисекундах.
92	`duration<double, micro>(t3 - t2).count() / Q,`	Среднее время одного интерполирующего поиска в микросекундах.
93	`n * sizeof(string), ok ? "OK" : "ОШИБКА");`	Дополнительная память интерполирующего поиска — размер копии массива в байтах (n · sizeof(string)); итог проверки: OK или ОШИБКА.
94	`}`	Конец цикла по номерам файлов.
95	`return 0;`	Завершаем программу.
96	`}`	Конец функции main.

3 Пример работы интерполирующего поиска

Отсортированный массив из 8 строк: apple, banana, cherry, grape, kiwi, lemon, mango, plum (индексы 0–7). Ищем строку lemon.

Шаг 1. low = 0, high = 7. Ключи: kl = key("apple") = 107 135 550 383 360, kh = key("plum") = 123 611 128 856 576, kx = key("lemon") = 119 182 883 515 904.

Доля: (kx − kl) / (kh − kl) ≈ 0,731. Позиция: pos = 0 + 0,731 · (7 − 0) = 5,1 → 5 (дробная часть отбрасывается).

a[5] = "lemon" — совпадает с искомой строкой. Элемент найден за один шаг. Двоичному поиску понадобилось бы 2 шага (позиции 3 и 5), последовательному в исходном массиве — до 8 сравнений.

Если бы a[pos] оказался меньше искомого, область сузилась бы до [pos + 1, high], если больше — до [low, pos − 1], и расчёт повторился бы для новой области.

4 Сводка оценок сложности

Алгоритм	Время (лучший / средний / худший)	Доп. память
Последовательный поиск	O(1) / O(N) / O(N)	O(1)
Сортировка копии (подготовка)	O(N·log N)	O(N)
Интерполирующий поиск	O(1) / O(log log N) / O(N)	O(1) сверх копии

Таблица — Оценки сложности алгоритмов

Вывод: при одном поиске выгоднее последовательный поиск, так как сортировка $O(N \cdot \log N)$ дороже одного прохода $O(N)$. При многократном поиске по одному массиву выгоднее интерполирующий: сортировка выполняется один раз, а каждый поиск занимает почти постоянное время. По результатам эксперимента ($n = 99\ 919$) сортировка окупается примерно начиная со 95 поисков.